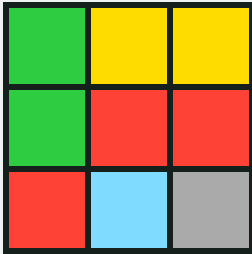


CORA Field Report

CORA is a program-induction reasoner for ARC puzzles that certifies every answer it gives. This report sets out what the program is trying to show, what it has already measured, what is running right now, and how the next result will be judged. Each part opens with the big picture and then gives the detail.



The evidence ladder drawn as an ARC grid, one cell per rung, in ARC's own colors.

STEP B RUN Running. No freeze marker yet.	PHASE COUNTER K2 proposals, 275 of 497	LAST QUANTITATIVE CHECK 19:48 UTC, 14 Sep 2026	GATE PROTOCOL v2 repaired, 67 of 67 checks. Full review pending, not frozen.
DEV BRANCH cora-tti-dev, clean and pushed			

BIG PICTURE

What CORA is trying to show

An ARC task shows a few input and output grids of colored cells and asks for the output of a new input. The rule is never stated. CORA answers by searching a typed language of grid programs, fitting a program to the demonstrations, and accepting it only if it survives leave-one-out re-induction: rebuilt from all demonstrations but one, it must still predict the one left out, for every fold. A CORA answer is therefore a program that can be checked, not a guess.

The research question sits one level above that engine. Better search and better priors have already been measured, and they help. What has never been shown is whether a reasoner can respond to its own failure by growing the language it reasons in, prove that the growth is real, and keep it only when it carries over to tasks that did not produce it.

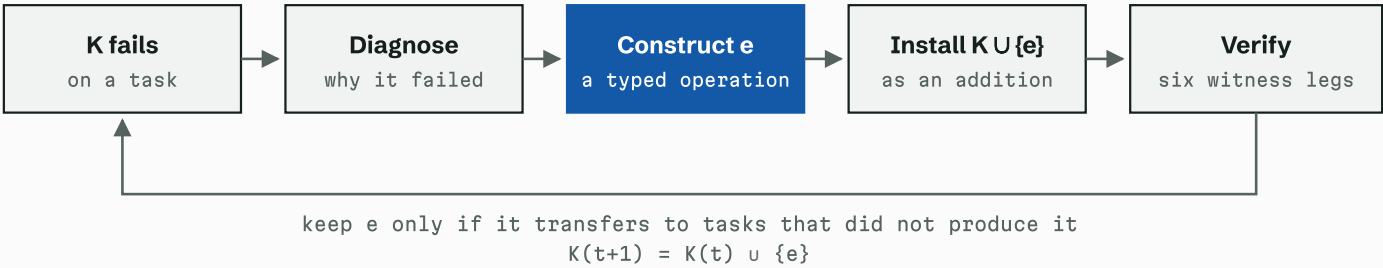
Training a model changes its parameters.

$$\theta \rightarrow \theta'$$

Capability growth changes the language. A new typed operation e joins the productions CORA can use.

$$K \rightarrow K' = K \cup \{e\}$$

The claim only means something if it is certified. An operation counts when the old language, searched completely within its bounds, could not reach the task; the new language does; the accepted answer actually uses the operation; re-induction from held-out demonstrations passes; the output is correct; and removing the operation destroys the gain.



The loop under test. Step B is the first run of the construct stage at scale. The Capability-Growth Gate is the verify stage.

Four levels of claim, never collapsed

Each level includes the one before it. The gate assigns them mechanically from recorded outcomes.

- L1

Generation-lane candidate
The frozen proposal lane emitted it. That licenses nothing about novelty. The lane's label, `NEW_SEMANTIC_PRODUCTION`, is never read by any rule.
- L2

Operational language extension
Added to K , it lets the fixed-budget search reach source tasks K could not reach. A macro of existing operations can do this, so L2 says nothing about new meaning.

- L3

Semantic expressivity extension, bounded

On a frozen probe set and under declared bounds, it produces behaviour that no program of the prior language, K_L4*, produces on the same inputs.
- L4

Semantic invention, bounded

L3, plus a full six-leg capability-growth witness on a transfer task that took no part in inventing it.

BIG PICTURE

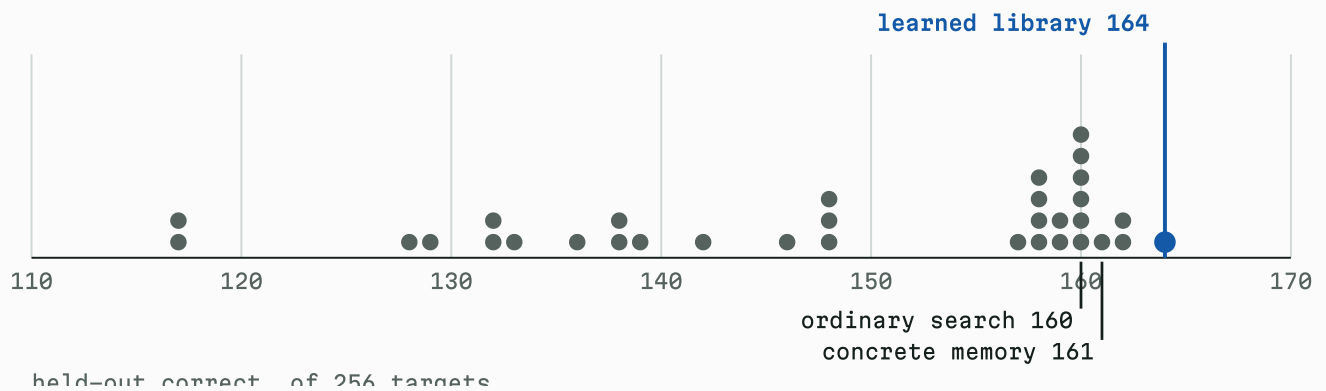
What the evidence says so far

The engine is real and certified on training tasks. Everything learned on top of it so far has made CORA choose better and search cheaper. None of it has made CORA solve a task it could not already solve.

MEASURE	VALUE	READING
Certified ARC training tasks	185 of 1000	The sealed result, version 23.
Public evaluation tasks certified	0 of 120	Nothing certified has carried over to the evaluation set yet.
Certified precision, training	40 of 42	A certified training answer is almost always correct.
Certified precision, evaluation development split	0 of 11	The same check carries almost no information off distribution.

The evidence ladder

RUNG	STATUS	WHAT ESTABLISHED IT
Search and prior transfer	 Yes	Structures found on source tasks make fresh tasks cheaper: 243,503 search units against 268,937 for ordinary search, over 256 targets.
Predictive-selection transfer	 Yes, narrowly	Learned knowledge changes which demonstration-consistent explanation is chosen. Removing it loses exactly 4 held-out successes.
Abstraction carries information	 Yes, narrowly	The learned library beat 32 of 32 matched controls that differ only in pseudo-random terminals.
Level 3A efficiency transfer	 Established	An earlier independent result, reinforced by the LAS-R1 costs.
Learned abstraction beats concrete memory	 No	The preregistered replication gave +3 with a 95 percent interval of -1 to 8, which includes zero.
Bounded search-reach gain	 No	Zero witnesses. Every policy solved the same 188 of 256 targets.
Level 3B capability transfer	 Not achieved	No case yet of baseline failure, success with the addition, use of it, passing leave-one-out, a correct output, and loss on removal.
Level 4 semantic self-extension	 Pending	The question Step B exists to answer.
Full-pipeline leave-one-out for LAS	 Not measured	No certification procedure was run for LAS, and none is claimed.



LAS-R1, 256 fresh targets. Each grey dot is one of 32 matched null libraries, identical to the learned library in masks, slot types and expansion sizes but with pseudo-random terminals. The learned library scored 164, above every null, which shows its invariants carry information. Concrete memory scored 161, and that 3-target margin did not survive the preregistered test.

DETAIL

The correction that changed the reading

An earlier test of test-time invention inside the real certified engine, on 60 development tasks, recorded 49 invention activations and zero acceptances by the certification gate. The plain engine and the invention arm both scored zero, and the invention arm used 1.57 times the wall time. For a while that looked like evidence that invention does not help. A read-only audit checked the code and found the test could not have shown a gain.

- 1. In `geocat_arc/object_reasoning/meta_induction.py` , `induce_computed_candidates` at line 550 checks its trigger before it consults installed concepts, so on many tasks the extension was never consulted.
- 2. When a concept did fit, the same function returned those programs at once. Ordinary search never ran, so the arm replaced K instead of adding to it.
- 3. `search_with_concepts` at line 378 omits the fold-coverability filter that ordinary `search` applies at line 455. Its only extra reach was exactly the programs the leave-one-out gate is built to reject.

That result is infrastructure evidence only. It is not evidence against extension, and the gate now requires a genuinely additive comparison.

BIG PICTURE, THEN DETAIL

What is running now: Step B

Step B is the blind invention run. Step A had already taken the failures CORA certifies on its training experience and grouped them into clusters by the shape of what was missing, never by task identity. Step B proposes candidate typed operations for each cluster from a frozen proposal language, installs them, and checks which ones resolve the failures under the unchanged verifier. Task identities stay hidden behind source tokens, and the transfer tasks are held out entirely.

Failure clusters	62	Grouped by frontier type, goal type and residual, never by task.		
Frozen candidates	4,784	816 new typed productions in the K2 lane, and 3,968 slot-learner repairs in the K1 lane.		
Declared phases	3	K2 proposals over 497 units; K1 proposals over about 11,900 searches; resolution of every class representative against every source.		
Run manifest	8476b211...	Every input, the runner and the candidate inventory are pinned. The runner refuses to start if any differs.		
K2 proposals		K1 proposals		Resolution

COUNTER READING ONLY. NO COMPLETION ESTIMATE IS MADE FROM IT.

Latest quantitative check

SIGNAL	READING AT 19:48 UTC, 14 SEP 2026
Runner	Alive , pid 106593 , 22 processes
Elapsed	18 days 15 hours
Phase	propose K2 , 275 of 497
Checkpoint journal	283 lines
Error, traceback or abort signatures	0
Final output hash file	Absent
STEP B FROZEN marker	Absent

Until the marker appears the run is watched by counts alone: process identity, the phase counter, journal line count, error signatures, and whether the final hash exists. No journal content, candidate, token mapping, transfer task or Lockbox item is read, and elapsed time is never treated as scientific progress.

RECORDED BEFORE THE OUTCOME

The runner's environment was captured from `/proc` on 14 September while it was still live. It sets `LANG` and `VIRTUAL_ENV` , and it does not set `PYTHONHASHSEED` or any thread limit. The gate will therefore carry `ENV-HASHSEED-UNSET-AT-RUN` as a standing caveat on the run. The recorded mitigation is the pre-run determinism gate, which passed on a masked comparison.

BIG PICTURE, THEN DETAIL

How the result will be judged

When Step B freezes, nothing is looked at until its output is pinned. Then eleven stages run in a fixed order. Every stage has an entry condition, a closed vocabulary

of outcomes, refusal conditions, and a stated claim it licenses alongside one it does not. The protocol is written and checked now, before any outcome exists, so no rule can be chosen after seeing a result.	
G1	Pin the output Preflight, then pin every Step-B artifact once and atomically, commit it, and bind the pin to its commit. needs STEP B FROZEN, the output hash file, and a stopped runner
G2	Pin everything that defines K A module pin over engine modules, the admissibility artifact, manifests and gate scripts, committed once. Nothing that affects K may change after this.
G3a	Prepare without looking Design source and pre-run records only. Additivity preflight and substrate checks on synthetic fixtures. Commit the tool manifest. this commit unlocks Step-B outputs
G3b	First look Check the declared schema assumptions, and count retained classes before reading any one of them.
G4	Characterize each candidate Typed interface, proposing sources, exact semantics, and bounded separation from K_{L4^*}.
G5	Assign rungs L1 to L4, mechanically. The generation label is never an input.
G6	Witness on the source tasks All six legs on the tasks that produced each extension, under an additive comparison.
G7a	Admit

No-regression and counterexample criteria, then admission, then commit the admitted set.

this commit unlocks the firewall and the Promotion extract

G7b **Resolve provenance**

Read the firewall for the first time. Leak and Promotion-regression checks may only withdraw. Commit the withdrawals.

this commit unlocks the E_transfer extract

G8 **Transfer, once**

E_transfer150 is opened a single time. The primary count N_w is the number of extension and task pairs outside the invention provenance whose six legs all pass.

G9 **Promote**

Only under the existing promotion protocol, with the promoting runner frozen inside its own manifest.

G10 **Analyse**

Rediscovery, automatic abstraction, K against K plus C, and ablation, with Level 3A and 3B reported separately.

G11 **Close**

Completion record, and Lockbox200 asserted untouched.

The six legs of a capability-growth witness

A witness is one extension on one task with all six legs passing. Every leg carries its own evidence object and exactly one status: `PASS` , `FAIL` , `INCONCLUSIVE` , `CHECKER_ERROR` , `RESOURCE_EXHAUSTED` or `NOT_MEASURED` . Only `PASS` counts, and nothing missing ever defaults to success.

B baseline_K_fails K, searched completely within its bounds, cannot reach the task. If any program K can find fits the demonstrations and gives the right output, K succeeded, even when it ranked a wrong program first.	P production_proposed The frozen generation record shows the extension was proposed by the declared mechanism.	U winner_uses_production With the extension added, the accepted program actually uses it.
L full_adaptive_L00_passes Every fold rebuilds everything that depended on the task's demonstrations, including the choice of the extension when the task helped invent it.	T final_output_correct The single accepted program, in one attempt, gets every licensed held-out output exactly right.	A ablation_without_e_fails An independent run with only the extension removed loses the gain, judged by the same reach rule as B.

An additive comparison, or no comparison

K and K plus e must differ only in e : the same solver, budget, ordering outside the extension, fitter, verifier, demonstrations and timeouts. A preflight on synthetic fixtures must show that the runtime never drops or reorders one of K's candidates when an extension is added. The old concept path is run through the same preflight as a negative control, and it must fail. A runtime that cannot pass records `ADDITIVE_INSTALL_PATH_UNAVAILABLE`, and the comparison is never weakened to fit it.

Admission, substrate, and the failure branch

An extension reaches the transfer set only if, on at least one task that produced it, the baseline fails, the proposal is on record, the winner uses it, and the ablation fails. Leave-one-out and output correctness on that task must pass, or be unmeasurable for a declared structural reason. Admission licenses measurement, not a claim.

A witness is claimed only on a runtime that passes every capability check on synthetic fixtures, in a fixed order, before any witness run. If none passes, the outcome is `WITNESS_SUBSTRATE_INCOMPLETE`. That is a recorded result, not permission to improvise another test after seeing a candidate.

If Step B yields no semantically separated, causally useful extension, the gate stops and records the negative as a negative about this constructor under these bounds. It does not license a return to memory, ranking or abstraction-selection work.

DETAIL

Provenance and sealed data

The gate's value depends on proving what was seen before the freeze, and that the bytes judged are the bytes the run produced. Both are enforced in code and covered by tests.

- **Readiness.** The one immutable pin can only be created when the freeze marker is present, the final output hash file exists, and no runner process remains, so the log cannot grow after it is hashed.
- **Content.** Artifact bytes are read only to hash them and count lines. They are never parsed, interpreted, printed, returned or written into the pin.
- **Binding.** After the pin is committed, a tracked record names its hash and commit. A pin deleted and recreated no longer matches that commit and stops the gate.
- **K is fixed.** A second pinned set covers everything that defines K, and every stage after G2 re-verifies both sets on every read.
- **Mixed files.** The raw ARC training file holds the Lockbox200 and transfer tasks alongside everything else, so no stage reads it. Tasks reach a stage only through per-split extracts.

What unlocks when

SEALED CLASS	HOLDS	OPENS
Step-B source	Runner, proposal inventory, frozen runtime	Stage 3
Pre-run records	Step A, admissibility, earlier concept records	Stage 3
Step-B outputs	Classes, inventory, witnesses, gate lanes, mechanism inputs	Stage 3, once the tool manifest is committed
Transfer provenance	The firewall holding transfer identities	Stage 7, once the admitted set is committed
Promotion data	The Promotion task extract	Stage 7, once the admitted set is committed
E_transfer	The E_transfer task extract	Stage 8, once withdrawals are committed
Lockbox	Lockbox200, holdout splits, every raw ARC data file	Never inside the gate
Never	The checkpoint journal, the withheld expectation seal	Never

Exposures before the freeze, recorded and quarantined

A scan of what earlier helper agents actually executed found more than their own summaries reported. None of it touched the running experiment or revealed what it proposed, kept or selected. All of it is recorded, and none of it may shape a rule, threshold, pool or substrate choice.

ID	WHAT HAPPENED	SCOPE
D1	One invention-corpus row decoded; its keys and demonstration count printed	Schema only
D2	The frozen candidate inventory listed by name and type	Pre-run search space, not results
D3	Runner and Step-B package source read	Design source, not outputs
D4	ARC evaluation inputs loaded while a predicate ran on 60 development tasks	Nothing scored
D5	Early dry runs hashed live files	Only pre-run gate lanes exposed
D6	Split documentation read	Zero identifiers
D7	Lockbox directory filenames listed	Names only
D8	Guard misclassified worktree copies and the firewall	Fixed before any access

DETAIL

The code

Two checkouts of one repository. The main checkout holds the live experiment and is never written by gate work. The dev worktree holds everything else, is pushed, and is never merged into main.

Main checkout

REASONING_PROJECT · BRANCH MAIN

scripts/cora_level4_stepB_run.py

The Step-B runner, pinned by its manifest and sealed until inspection.

scripts/restart_stepB_after_reboot.sh

The only permitted way to relaunch Step B. It resumes from the journal.

level4_stepB/

The frozen proposal inventory, probe-witness generator and install path.

level4_blind_runtime/

Dev worktree

REASONING_PROJECT_TTI · BRANCH CORA-TTI-DEV

geocat_arc/object_reasoning/

The certified engine: the typed meta-language, induction with the leave-one-out gate, concept search, and the v2l engine.

cora_tti/freeze_pin.py

Pin readiness, the atomic pin, commit binding and module-pin verification.

cora_tti/gate_guard.py

Sealed classes, unlock events, provenance checks and the access ledger.

The frozen typed runtime that defines the baseline language K_L4*.

outputs/cora_breakthrough/

Run artifacts, manifests, the checkpoint journal and the firewall.

logs/level4_stepB_run.log

The one file watched for the freeze marker.

RESUME_STAGE1.md

The running project log, local and untracked.

cora_tti/scoped_slot_fitting.py,
meta_baseline.py, abstraction_lattice.py

The fitter, the identified baseline reasoner and the abstraction lattice behind the synthetic studies.

cora_tti/tfg_extractor.py, tti_loop.py,
ablation_ledger.py

Typed failure graphs, the test-time loop and the witness ledger.

cora_parent/interfaces.py

The frozen failure-cause vocabulary and component contracts.

scripts/pin_stepB_freeze.py,
gate_preflight.py, gate_capture_run_env.py,
check_gate_protocol_v2.py

Command-line gate tooling.

docs/

Protocols, results, the evidence ladder and the deviation record.

DETAIL

How to run it

Run the gate tooling tests

All four suites use synthetic fixture trees and never touch the live run.

```
cd /deltos/e/lesion_phes/code/python/pipeline/Reasoning_Project_tti
/home/cnptp/.venvs/lesegen/bin/python -m pytest -q -p no:cacheprovider \
  tests/test_stepB_freeze_pin.py tests/test_gate_guard.py \
  tests/test_gate_preflight.py tests/test_gate_capture_run_env.py
```

Check Step B, by counts only

```
cd /deltos/e/lesion_phes/code/python/pipeline/Reasoning_Project
pgrep -af cora_level4_stepB_run.py | head -1
tail -c 300 logs/level4_stepB_run.log | tr '\r' '\n' | tail -1
wc -l < outputs/cora_breakthrough/level4_stepB_journal.jsonl
grep -c 'STEP B FROZEN' logs/level4_stepB_run.log
test -f outputs/cora_breakthrough/level4_stepB_output_hash.txt && echo hash-present
```

At the freeze marker, before looking at anything

Every command prints `OUTCOME <name>` first, with a fixed exit code per outcome.

```
cd /deltos/e/lesion_phes/code/python/pipeline/Reasoning_Project_tti
export PYTHONHASHSEED=0 OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1
python3 scripts/gate_preflight.py # expect PREFLIGHT_OK
python3 scripts/pin_stepB_freeze.py # PIN_CREATED, or a not-ready outcome
git add -f outputs/tti/stepB_gate/pin
git commit -m "Pin the frozen Step B output"
python3 scripts/pin_stepB_freeze.py --bind # expect PIN_BOUND
git add docs/stepB_gate/pin_binding.json
git commit -m "Bind the Step B freeze pin to its commit"
python3 scripts/pin_stepB_freeze.py --require-binding
```

Verify at any later point

```
python3 scripts/pin_stepB_freeze.py --require-binding
python3 scripts/pin_stepB_freeze.py --verify-modules
python3 scripts/check_gate_protocol_v2.py docs/CORA_CAPABILITY_GROWTH_GATE.md
```

If the Step-B runner dies

```
cd /deltos/e/lesion_phes/code/python/pipeline/Reasoning_Project
bash scripts/restart_stepB_after_reboot.sh
```

The certified engine harness

From the repository README. Not while Step B holds the machine's cores.

```
python3 scripts/run_unified_harness.py --workers 16 --out-dir outputs/run --run-id repro
python3 scripts/paper_tables.py
```

DETAIL

What has been tested, and what is still in flight

STUDY	QUESTION	WHAT IT FOUND
Evidence-closure census	Were the constructive positives real and reproducible?	The original census could not be reproduced because it used an unseeded hash. The corrected census admitted 13 distinct targets in 896 attempts.
Failure-signal study	Does ranking proposals by their recorded failures help?	No. Held-out correctness fell from 15 of 24 to 7 or 8 of 24 at about three times the cost, and the real failure trace beat a shuffled one only 8 against 6.
Acquisition-and-transfer cycle	Can a constructed schema do useful work on tasks that did not create it?	Search improved on 4 of 22 targets. Operational growth was weak, and no semantic growth was claimed.
Prospective abstraction transfer	Do abstractions formed from discovered programs beat concrete schemas?	No. Concrete-schema reuse beat both abstraction rules on every measure, and one rule was indistinguishable from its unlearned control. A held-out prediction rescue did reproduce prospectively, and was necessary under ablation.
LAS-v1	Can CORA choose the useful level of abstraction from source evidence alone?	Yes on correctness, no on cost: 16 against 14 held-out successes over 24 targets, a two-target margin that is not a resolvable effect, at about 2.8 times ordinary search's total cost.
LAS-R1	Does LAS-v1 replicate on 256 new targets?	Not robustly. It beat all 32 matched nulls and was cheapest, but its +3 over concrete memory has an interval that includes zero. The synthetic line is closed.
Real-engine invention arm	Does test-time invention inside the certified engine help on development tasks?	Not testable as built. 49 activations, zero acceptances, and a pass@2 of zero in both arms, but the concept hook replaced ordinary search on a hit, so the arm could not have shown a gain.

Gate tooling

SUITE	TESTS	WHAT IT PROVES
Freeze pin	51	Refuses early pins, never overwrites, catches drift, deletion, growth and repinning, and leaks no content.
Access guard	42	Sealed data opens only at its stage and committed event, and provenance failures stop the gate.
Preflight	10	The hash seed, thread limits, committed tooling and a clean worktree are enforced.
Environment capture	6	Only allowlisted values ever reach the record.

All 109 ran under a Python audit hook that recorded zero touches of the live experiment tree. A canary confirmed the hook catches a touch it is told to forbid.

Still in flight

- **Step B itself.** The only experiment that can move the Level 4 rung.
- **A full independent review of protocol v2.** An earlier review saw only a fragment of the document. After this review the protocol is committed, hashed and frozen.
- **Gate machinery not yet built.** The separation certificate, the six-leg witness runner, the adaptive leave-one-out harness, final-output and ablation instrumentation, the additivity preflight, the split extractor and the stage scripts. The protocol permits building candidate-independent pieces between the pin and the first look.

BIG PICTURE

Direction

If Step B yields a certified extension

- Turn the one-off into a loop: fail, construct, verify, transfer, retain, repeated over many tasks.
- Accumulate several genuine capability-growth witnesses before building

If it does not

- Record the negative as a negative about this constructor under these bounds.
- Do not return to memory, ranking or abstraction selection. That line has a documented ceiling.

anything learned.

- Only then train a controller that predicts which kind of extension to construct.

- Redesign only the semantic constructor: a typed meta-constructor that builds bridges such as `Set[Region] -> Grid`, `Relation[A, B] -> Mapping[A, B]` and `Collection[A] -> Aggregate[B]`, under a new pre-registered protocol and a fresh holdout.

Either way

Certification has to hold off distribution. It is right 40 times in 42 on training tasks and 0 times in 11 on the evaluation development split. Until a passing leave-one-out check means something on unseen task families, no invention result will carry to the real benchmark. Nothing currently addresses this, and working on it needs development and training runs that the current directive holds until the Step-B verdict.

The calendar is tight. Project notes record a Kaggle submission on 2 November and a paper on 8 November, with a bar of more than 200 certified training tasks and more than zero on evaluation before Kaggle. The only capability-growth experiment waits on a run with no completion estimate.

REFERENCE

Terms

ARC

The Abstraction and Reasoning Corpus: grid puzzles given as a few demonstrations with a hidden rule.

e

A candidate extension: one new typed operation.

Leave-one-out re-induction

K

CORA's current operational language, the typed productions its search may use.

K_L4*

The 11 grounded productions that existed before Level 4 and are reachable under the frozen search. The baseline for separation.

Step A and Step B

Rebuild the answer from all demonstrations but one and predict the one left out, for every fold.

K1 and K2 lanes

Slot-learner repairs, and new typed productions. Only K2 can ever yield an extension.

E_invent and E_transfer150

The 450 and 150 Experience tasks reserved for invention and for transfer.

LAS

Learned abstraction selection, the synthetic study line now closed.

Step A extracted certified failure frontiers into 62 clusters. Step B is the blind invention run over them.

Experience, Promotion, Lockbox

The 600, 200 and 200 task splits of the 1000 ARC training tasks.

Freeze pin and module pin

The immutable digest records of Step B's artifacts and of everything that defines K.

Typed failure graph

A structured record of why a search failed, built from the real search trace.

Compiled from docs/CORA_EVIDENCE_LADDER.md , docs/CORA_LAS_R1_RESULT.md , docs/CORA_SELF_EXTENSION_LOOP_AUDIT.md , docs/CORA_STEPB_PREFREEZE_DEVIATIONS.md , docs/LOCKBOX_PROTOCOL.md , docs/CORA_LEVEL4_MANIFEST.md , the staged gate protocol v2, and the project log.

Nothing sealed was read to write this page. The 185 of 1000 figure is the single sealed ARC number, and no other score is implied.